

California Science and Technology University

DevOps with AI — Syllabus (Revised for Agentic AI)

CSE636 · Summer 2026

Revision note (v2): This edition modernizes the course around the 2025–2026 shift from *AI-assisted* DevOps to *agentic* DevOps — autonomous AI agents that plan and execute multi-step work — while preserving the original 7-week / 13-session structure, grading, and policies. Sessions, labs, assignments, and references have been updated. The course uses **no required (purchased) textbook**: required readings are free, continuously updated online references, with optional recommended books for students new to DevOps or AI (see *Textbook and Reference Material*).

Course & Faculty Information

Lecturer	Qingsong Zhang
E-mail	qingsong.zhang@cstu.edu
Time	Wednesday 7:30pm – 9:30pm (Online) · Saturday 10:40am – 12:10pm (Onsite)
Contact	23 Hours
Credit	1.5 Units
Office hours	By Appointment
Review Date	6/8/2026

Course Description

DevOps with AI (1.5 units)

This course explores how artificial intelligence (AI), machine learning (ML), and — increasingly — **autonomous AI agents** can be integrated into DevOps practices to automate, optimize, and enhance software development and operations. Students move beyond using AI as a passive assistant to designing **agentic systems** that perceive, plan, use tools, and act across the software delivery lifecycle, with appropriate guardrails and human oversight.

Students will gain hands-on experience with AI-assisted and agentic CI/CD pipelines, AI coding agents, the Model Context Protocol (MCP), predictive analytics, intelligent monitoring and observability, autonomous incident response (agentic SRE), and AI security and governance.

By the end of the course, students will be able to:

1. Explain how AI agents and ML augment and automate DevOps processes across increasing levels of autonomy (assistant → human-in-the-loop → human-on-the-loop → autonomous).

2. Design and integrate agentic automation into CI/CD pipelines, including MCP-connected tools and AI coding agents.
 3. Use predictive analytics and forecasting for reliability, performance, and cost (FinOps) improvements.
 4. Build AI- and agent-driven monitoring, anomaly detection, root-cause analysis, and autonomous incident response.
 5. Evaluate AI agents, frameworks, and protocols for DevOps at scale — applying security, governance, and responsible-AI guardrails.
-

Textbook and Reference Material

Textbook Information

There is no required textbook to purchase. Agentic DevOps is a fast-moving field (2025–2026) with no single authoritative textbook; any printed book is partly outdated before it ships. Instead, this course uses a **curated set of freely available, continuously updated references** (listed below), assigned per session. All required readings are free and accessible online.

For students who are **new to DevOps and/or AI** and want a deeper anchor for the fundamentals, the following books are **optional and recommended** (not required, not purchased for any exam or assignment):

- **DevOps foundations** — *The Phoenix Project* (Kim, Behr, Spafford) for an approachable narrative on *why* DevOps exists, or *The DevOps Handbook, 2nd ed.* (Kim, Humble, Debois, Willis) for the practices.
- **AI / LLM foundations** — *AI Engineering: Building Applications with Foundation Models* (Chip Huyen, O'Reilly, 2025) for LLMs, RAG, agents, and evaluation.
- **Reliability foundations** — *Site Reliability Engineering* (Google) — **free online** at <https://sre.google/books/> — for the SRE concepts behind several later sessions.

Reference Information (current — required readings)

Primary, freely available references that reflect the current state of agentic DevOps. These are the course's working "textbook," assigned per session:

1. **Anthropic — "Building Effective Agents"** (engineering guide): <https://www.anthropic.com/engineering/building-effective-agents>
2. **Anthropic — Claude Developer & Claude Code documentation:** <https://docs.anthropic.com>
3. **Model Context Protocol (MCP)** — specification and SDKs: <https://modelcontextprotocol.io>
4. **OpenAI Platform documentation:** <https://platform.openai.com/docs>
5. **Google — "Agents" / Agent Development Kit (ADK)** documentation: <https://google.github.io/adk-docs/>
6. **OpenTelemetry — GenAI semantic conventions** (observability for AI/agents): <https://opentelemetry.io/docs/specs/semconv/gen-ai/>
7. **DORA — Accelerate State of DevOps / DevOps metrics:** <https://dora.dev>
8. **Platform Engineering & Internal Developer Platforms** (Backstage): <https://backstage.io> · <https://platformengineering.org>
9. **Open Policy Agent (Policy-as-Code):** <https://www.openpolicyagent.org>

10. **Google — Site Reliability Engineering** (free online book): <https://sre.google/books/> — SRE concepts behind the monitoring, incident-response, and agentic-SRE sessions.
 11. **GitHub Copilot, Cursor, and AIOps vendor docs** (Datadog Bits AI, New Relic AI, Dynatrace Davis, PagerDuty AI) — linked per session.
-

Prerequisite

Working experience with Python programming, basic computer concepts, and business software tools such as Anaconda. Familiarity with Git and containers (Docker) is recommended; no prior ML or LLM experience required.

Instructional Method

A combination of lectures, class participation, and hands-on work facilitates learning. Projects are assigned individually or as a group. Homework includes course readings, assignments, and course projects. Students are encouraged to seek instructor assistance, engage in group discussion, and consult the school advisor.

Responsible AI-tool use: Students are expected to *use* AI coding agents and assistants in labs and projects, and to **disclose and critically evaluate** their use — including verifying correctness, reviewing agent-generated changes, and noting failures. Uncritical or undisclosed reliance on AI output is treated under the Academic Integrity policy.

Out of Class Work

Students are expected to engage in a minimum of two hours of out-of-class work for every hour of classroom instruction. Out-of-class work includes course readings, group projects, working on course problems or case studies, exam preparation, course research, and group discussions. Out-of-class work helps solidify learning and prepare for the next lecture. It is assessed and graded and impacts the student's final grade.

Learning Resource Center

Students are required to use the library for study. The librarian or library assistant is available to provide assistance and guidance on navigating the online resources of the Learning Resource Center. The CSTU website provides detailed instructions on accessing the Learning Resource Center and searching for articles in a specific field.

Objectives

To accomplish its mission, CSTU is committed to enhancing student competencies by:

- Providing working adults with higher educational opportunities that are flexible and accessible.
- Providing graduate-level students professional training and preparation for careers.
- Providing higher educational opportunities that are current with technology and career demands.

- Providing faculty members with demonstrated expertise in their respective domain, both professionally and academically.
- Integrating into the educational process a better understanding of cultural diversity needs.
- Delivering educational support services that meet student life demands and schedules.
- Building within students a value for life-long learning and education.
- Providing educational resources in a manner that effectively uses current technology.

CSTU is committed to the highest ethical standards in pursuit of its mission. These values are honored in our daily structure and activity as members of this community. We respect the rights and dignity of others, conduct ourselves with integrity, are accountable as individuals and community members for ethical conduct and compliance with applicable laws, University policies, and directives, and conscientiously strive for excellence in our work.

Student Learning Outcomes

The University Student Learning Outcomes assessed and reinforced in this course include but are not limited to:

- Communication
 - Critical Thinking
 - Information Literacy
 - ML, AI, and **AI-agent / agentic-systems** fundamentals
-

Attendance Policy

The University recognizes the correlation between attendance and both student retention and achievement. Per University Policy, **students are expected to attend all classes, actively participate, and complete all assigned course work for all courses for which they are registered.**

Evaluation

Assessment may be determined by participation, out-of-class work, quizzes, tests, projects, and presentation of the project.

Evaluation Criteria & Grading Scale

Component	Weight
Mid-term Exam	15%
Group Project	20%
Case Discussions	30%
Final Exam	20%
Class Participation	15%
Total	100%

Percent	Grade	Percent	Grade
98%–100%	A+	78%–79%	C+
93%–97%	A	73%–77%	C
90%–92%	A-	70%–72%	C-
88%–89%	B+	68%–69%	D+
83%–87%	B	63%–67%	D
80%–82%	B-	60%–62%	D-
		Below 60%	F

WEEKLY SCHEDULE

Week 1: Foundations of AI-Assisted & Agentic DevOps

Session 1: From DevOps to Agentic DevOps

1. Evolution of practice: DevOps → DevSecOps → AIOps → **Agentic DevOps**, and why agents matter now
2. Assistants vs. agents vs. autonomous systems — the **levels of autonomy** and where humans stay in/on the loop
3. Anatomy of an AI agent: perception, planning/reasoning, tool use, memory, and the act–observe loop
4. Key challenges in modern DevOps that agents target (toil, scale, MTTR, cognitive load)
5. Industry landscape & case studies: Claude Code, GitHub Copilot agent mode, Cursor, Devin

Session 2: AI/ML & LLM Foundations for DevOps

1. ML categories recap (supervised, unsupervised, reinforcement) and where LLMs/foundation models fit
2. LLM capabilities that enable agents: reasoning, **tool/function calling**, structured output
3. DevOps data sources (logs, metrics, traces, configs) as agent context; **RAG** for operational knowledge
4. Prompting, **context engineering**, and evaluation basics
5. How agents connect to DevOps toolchains via APIs, CLIs, and **MCP**

Lab (Week 1):

1. Set up a cloud-based DevOps lab (AWS/Azure/GCP)
2. Run an AI coding agent (e.g., Claude Code or Copilot) against a sample repository; collect CI/CD and system-monitoring data for later agent tasks

Assignment:

- Write a short report on 3 real-world agentic-DevOps deployments. For each, identify the level of autonomy, the tools/data the agent uses, measured impact, and the main risks.

Week 2: AI Agent Tooling, Protocols & Platforms

Session 3: AI Coding Agents & Agentic Tooling

1. AI coding agents compared: Claude Code, Cursor, GitHub Copilot (agent mode), Devin, OpenAI Codex
2. AIOps platforms with agentic features: Datadog Bits AI, New Relic AI, Dynatrace Davis, PagerDuty AI
3. Capabilities: autonomous code changes, anomaly detection, prediction, optimization
4. Strengths, weaknesses, and **when not to use an agent**
5. Building toolchains with AI/agent plug-ins

Session 4: Agent Protocols, Frameworks & Environments

1. **Model Context Protocol (MCP)**: servers, tools, resources — connecting agents to DevOps systems safely
2. Agent orchestration frameworks: LangGraph, CrewAI, AutoGen, Google ADK
3. Cloud agent services: Amazon Bedrock Agents, Vertex AI Agent Builder, Azure AI Foundry
4. Secrets, credential, and **permission management for agents** (least privilege, scoped tokens, sandboxing)
5. Lab walkthrough: building a basic MCP-connected agent

Lab (Week 2):

1. Deploy a Jenkins or GitHub Actions pipeline that invokes an AI agent for code review / quality analysis; connect a simple MCP server exposing a repo or build tool.

Assignment:

- Compare at least two agent frameworks or AI coding agents and propose an integration **and governance** plan for a mid-sized software team.
-

Week 3: Agentic CI/CD Pipelines

Session 5: AI & Agents in Code Quality and Testing

1. Agentic code review and ML-assisted static analysis
2. AI-driven test generation, self-healing tests, and test prioritization
3. Reducing flaky tests; agents that diagnose and fix failures
4. Measuring coverage and quality with AI; **eval harnesses for AI-generated code**
5. Tools and frameworks for AI-enhanced QA

Session 6: Self-Optimizing & Self-Healing Pipelines

1. Agents that triage and fix failing builds and open pull requests
2. Using ML to reduce pipeline execution time; dynamic build-resource allocation
3. Predictive build-failure detection
4. Caching and artifact management with AI insights
5. Continuous feedback loops and **guardrails on autonomous merges** (approval gates, blast-radius limits)

Lab (Week 3):

1. Implement an agent that detects a failing CI build, proposes a fix, and opens a PR behind a human-approval gate.

Assignment:

- Build a CI pipeline where an agent skips unnecessary builds/tests *and* auto-remediates one common class of failure.
-

Week 4: Predictive Analytics & Capacity Intelligence**Session 7: Risk Prediction & Deployment Intelligence**

1. Predicting failed deployments from historical data; progressive delivery (canary / blue-green) with AI gating
2. Agentic rollback decisions and recommendations
3. ML models for deployment success scoring and change-risk
4. Data-labeling and evaluation challenges
5. Industry case studies

Session 8: Capacity & Performance Forecasting

1. Forecasting infrastructure usage; **FinOps and AI-driven cost optimization**
2. Identifying bottlenecks; performance prediction for microservices
3. ML-driven autoscaling (predictive HPA, KEDA)
4. Time-series forecasting algorithms
5. Agents that recommend and apply scaling policies

Lab (Week 4):

1. Build and train a time-series forecasting model to predict CPU and memory usage; wire the predictions into an autoscaling decision.

Assignment:

- Create a report analyzing the effectiveness of AI forecasting for autoscaling in Kubernetes clusters, including cost impact.
-

Week 5: Intelligent Monitoring, Observability & Agent Telemetry**Session 9: AI-Driven Anomaly Detection**

1. Using ML on logs, metrics, and traces for anomalies
2. Unsupervised approaches (clustering, isolation forests) and LLM-based detection
3. Real-time vs. batch detection; noise reduction and false positives
4. Best practices for integrating detection in pipelines
5. **Observability for AI agents:** OpenTelemetry GenAI semantic conventions — tracing agent actions, tokens, cost, and latency

Session 10: Root Cause Analysis with AI Agents

1. Graph-based algorithms for dependency mapping
2. Log and trace correlation using LLMs and NLP
3. AI-driven alert grouping and incident summarization
4. Prioritizing incidents based on impact
5. **Agentic RCA**: investigate-and-report agents that automate post-incident analysis

Lab (Week 5):

1. Implement log/trace anomaly detection using unsupervised learning on sample data, and instrument an agent with OpenTelemetry GenAI conventions.

Assignment:

- Build a system that detects anomalies and produces an AI-generated root-cause summary with prioritized alerts.
-

Week 6: Autonomous Incident Response & Agentic SRE

Session 11: Self-Healing Systems & Agentic SRE

1. Self-healing architecture and AI as a "virtual SRE"
2. Autonomous rollback and failover with **guardrails and approval gates**
3. Agent reasoning loops (ReAct, reflection) and reinforcement learning for response
4. Orchestration with Kubernetes Operators and agents
5. Levels of autonomy and **blast-radius control**

Session 12: Alert Triage, Runbooks & ITSM Integration

1. Intelligent alert suppression and correlation-based incident grouping
2. **Agentic runbooks/playbooks** for triage
3. Integration with ITSM and on-call tools (PagerDuty, ServiceNow, Opsgenie)
4. Tracking remediation effectiveness; AI-assisted postmortems
5. On-call augmentation and escalation policies

Lab (Week 6):

1. Create a Python-based agent (with MCP-exposed tools) that triages a simulated incident and triggers a *gated* automated remediation.

Assignment:

- Implement a self-healing mechanism for a microservice using agent-driven triggers with a human-approval step.
-

Week 7: Agentic IaC, Platform Engineering, Security & Governance

Session 13: Generative/Agentic IaC, Platform Engineering & AI Governance

1. Generating and refactoring Terraform / OpenTofu / CloudFormation with agents; **Policy-as-Code** (OPA)
2. Internal Developer Platforms (Backstage) and golden paths augmented by AI
3. **Agent security:** prompt injection, tool-permission scoping, sandboxing, secret handling
4. Supply-chain security (SLSA), audit trails, compliance, and responsible-AI governance
5. Capstone overview: an end-to-end agentic DevOps pipeline

Assignment / Capstone:

- Implement an agent-augmented IaC module and present the end-to-end agentic DevOps project (pipeline → deploy → observe → auto-remediate, with guardrails and governance).

Academic Integrity

CSTU expects that all academic work submitted by students be original, or — in the case of cited material — properly acknowledged as the work, ideas, or language of another. All acts of academic dishonesty are strictly prohibited. These include, but are not limited to, cheating, plagiarism, fabrication, unauthorized collaboration, misappropriation of resource material, or any other violation of University regulations.

On AI tools: Use of AI coding agents and assistants is permitted and expected in designated labs and projects, but must be **disclosed**. Students remain fully responsible for the correctness, security, and originality of submitted work, including any AI-generated content. Submitting AI output as one's own unverified work, or using AI where prohibited (e.g., exams), constitutes a violation of this policy.